Sanjivani Rural Education Society's

Sanjivani College of Engineering, Kopargaon-423603

(An Autonomous Institute Affiliated to Savitribai Phule Pune University, Pune)

NAAC 'A' Grade Accredited

Department of Information Technology

UG Programme in IT NBA Accredited

A MINI PROJECT REPORT ON

"SMART SMS / MESSAGE DETECTOR SYSTEM USING NATURAL LANGUAGE PROCESSING "

Submitted by

Komal Hembade [UIT23F1065]
Atharv Hon [UIT23M1066]
Saurabh Jadhav [UIT23M1067]
Anita Jadhav [UIT23F1068]

Under the Guidance of

Dr. S. A. Thanekar

Department of Information Technology

Sanjivani College of Engineering, Kopargaon-423603

Academic Year 2026–2027

1. Introduction and Problem Statement

Short Message Service (SMS) remains a widely used channel of digital communication, which has made it an attractive route for unwanted promotional messages and fraudulent "smishing" attempts. SMS text is short, informally written and carries little context, and manual inspection or static keyword/rule-based filters do not scale because spammers continuously vary their wording. Natural Language Processing (NLP) combined with deep-learning classifiers can learn the distinguishing patterns of spam directly from labelled data and generalise to unseen messages.

1.1 Problem Statement

To study, verify and document an AI-based system capable of automatically classifying an input SMS message as spam or ham using NLP-based text preparation and deep learning classification models, and to report the system's actual, verifiable technical behaviour and performance rather than only its documented claims.

1.2 Source of the Studied Implementation

The data-preparation pipeline, the Dense/LSTM/Bi-LSTM notebook, the trained model files and the Streamlit application studied here are adapted from the public GitHub repository "deepankarvarma/SMS-Spam-Detection-using-NLP" [6] and are not independently developed from scratch. The contribution of this work is the independent study, verification, testing and documentation of that codebase, including cross-checking the README's claims against the notebook's own printed outputs and saved model files.

2. Objectives

- To study SMS spam detection as a binary NLP text-classification problem and analyse the UCI SMS Spam Collection dataset, including its class imbalance.
- To verify the exact preprocessing, tokenization and sequence-padding pipeline, and the architecture, hyperparameters and training configuration of the Dense, LSTM and Bi-LSTM models actually saved in the project.
- To report the models' evaluation results as printed by the notebook and compare them with the accuracy figures claimed in the project README.
- To examine the Streamlit application (app.py) and document its real behaviour, including implementation limitations.
- To identify and record discrepancies between the project's documentation and its verifiable code and outputs.

3. Proposed Methodology and System Architecture

The end-to-end pipeline is: load SMSSpamCollection → inspect class counts → down-sample the ham class → train/test split → fit a Keras Tokenizer on training messages → convert to padded integer sequences → train Dense, LSTM and Bi-LSTM models (and a comparison SVM) → evaluate on the test split → save the models as .h5 files → load a saved model in the Streamlit app for live prediction (Figure 1).

3.1 Dataset and Preprocessing

The UCI SMS Spam Collection [1] contains 5,574 raw lines, of which 5,572 are parsed: 4,825 ham and 747 spam messages (spam ≈ 15.48% of ham). To handle this imbalance, ham is randomly down-sampled to 747 messages (random_state = 44), giving a balanced set of 1,494 messages, split 80:20 (random_state = 434) into 1,195 training and 299 testing messages. Text is lower-cased by the Keras Tokenizer, restricted to the 500 most frequent words plus an <OOV> token, and padded/truncated (post) to a fixed length of 50 tokens. No stopword removal, stemming or lemmatization is implemented.

3.2 Models and Training

All neural models start with a trainable Embedding(500, 16) layer. The Dense model (10,017 parameters) applies GlobalAveragePooling1D, two Dense hidden layers (48 and 24 units, ReLU) with Dropout(0.2) after each, and a Dense(1, sigmoid) output. The LSTM model (14,261 parameters) uses two stacked LSTM layers (20 units each) and the Bi-LSTM model (13,961 parameters) uses one Bidirectional LSTM layer (20 units), each followed by Dense(1, sigmoid). An SVM (RBF kernel, C = 10, gamma = 0.1) is trained on the same padded sequences for comparison. Models are compiled with binary cross-entropy and the Adam optimiser and trained with EarlyStopping on val_loss (default batch size 32); training ran for 25, 21 and 13 epochs for Dense, LSTM and Bi-LSTM respectively. The test split is also used as validation data, and no separate third split exists.

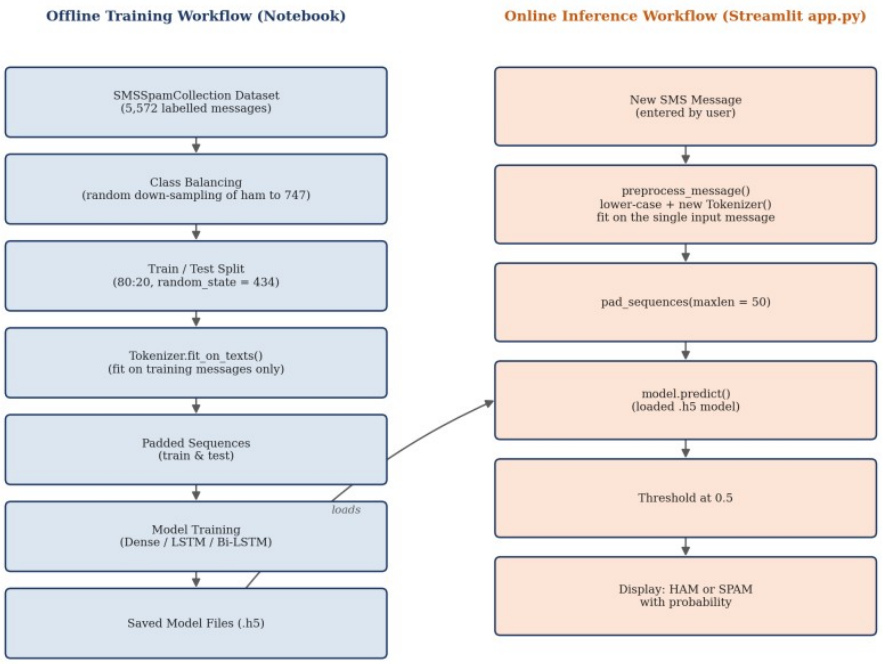


Figure 1: Overall Workflow (Offline Training and Online Inference)

4. Technologies and Tools Used

- **Language and environment:** Python; Jupyter/Kaggle notebook (development and training); GitHub (base repository).
- **Frameworks:** TensorFlow 2.12.0 / Keras (Embedding, GlobalAveragePooling1D, Dense, Dropout, LSTM, Bidirectional layers); Streamlit 1.21.0 (web interface); scikit-learn (train_test_split, SVC).
- **Libraries:** NumPy 1.23.5; pandas, seaborn/matplotlib and wordcloud (notebook only, not pinned in requirements.txt).
- No database, external API, backend framework or cloud AI service is used anywhere in the project.

5. Implementation and Working

During training, the notebook loads and balances the dataset, tokenizes and pads the messages, trains the three Keras models and the SVM, and saves each Keras model as a .h5 file. The Streamlit application (app.py) loads Dense_Spam_Detection.h5 once at start-up and, for each message entered through st.text_input(), performs the following inference steps: (1) lower-case the message; (2) create a Tokenizer(num_words=10000) and fit it on the message; (3) convert to a sequence and pad to length 50; (4) call model.predict(); (5) display “spam” if the probability exceeds 0.5, otherwise “ham”, together with the probability percentage. Empty input produces no prediction.

5.1 Verified Implementation Limitations

- **Tokenizer mismatch:** preprocess_message() fits a new Tokenizer on the single input message instead of reusing the tokenizer fitted on the training data, so the word-to-index mapping is inconsistent with the vocabulary the Embedding layer was trained on. No training-time tokenizer is saved in the project.
- **Deployed model:** the application uses the Dense model, which has the lowest notebook accuracy of the three neural models.
- **Notebook vs. saved models:** the saved .h5 architectures (e.g., two hidden Dense layers) differ from the currently visible notebook cells, indicating the notebook was edited after training.
- app.py has no error handling, authentication, input sanitisation beyond lower-casing, or storage of submitted messages.

6. Results and Discussion

Table 1 lists the test results exactly as printed by the notebook's own model.evaluate() / .score() calls on the 299-message test split; these are not new experiments.

Model	Test Accuracy	Test Loss	Epochs Run
Dense	93.65%	0.1238	25 of 50 configured
LSTM	95.32%	0.1618	21 of 30 configured
Bi-LSTM	94.98%	0.1490	13 of 30 configured
SVM	69.57%	Not printed	N/A

Table 1: Notebook-Reported Model Performance

LSTM achieved the highest notebook-reported accuracy (95.32%), narrowly ahead of Bi-LSTM (94.98%) and the Dense network (93.65%); since Bi-LSTM stopped after only 13 epochs, the small gap may partly reflect early stopping rather than architecture. The SVM's low accuracy (69.57%) is consistent with training it on raw token-ID sequences, which are not a meaningful continuous feature space. The project README claims 98.5% (Dense), 98.6% (LSTM), 98.8% (Bi-LSTM) and 97.6% (SVM), and states that Bi-LSTM performed best; neither claim is reproduced by the notebook's own outputs (Figure 2). Precision, recall, F1-score and a confusion matrix are not computed anywhere in the project and are therefore not reported.

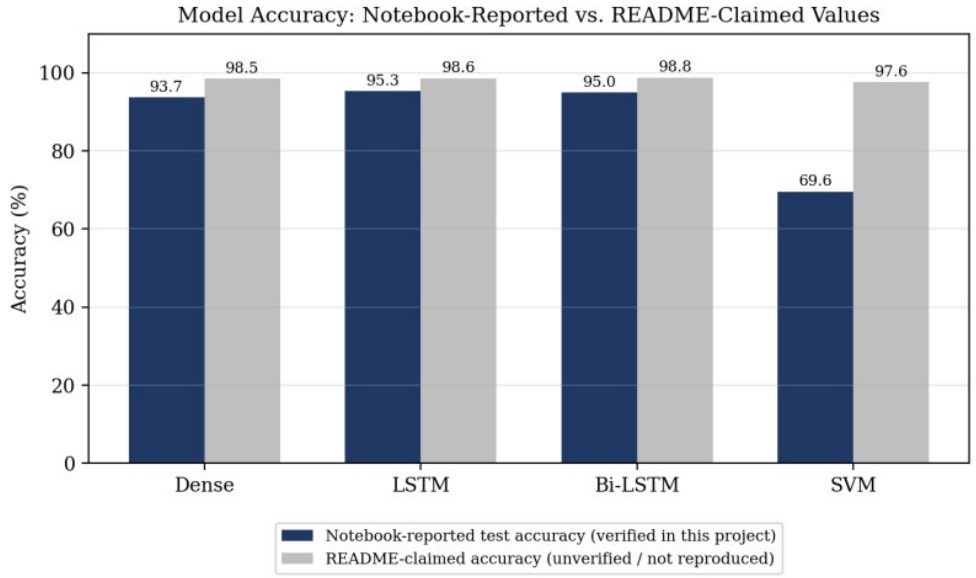


Figure 2: Model Accuracy — Notebook-Reported vs README-Claimed Values

7. Conclusion and Future Scope

The verified evidence shows that an NLP-preprocessing-plus-neural-classifier pipeline can distinguish spam from ham SMS messages well above chance on the UCI SMS Spam Collection, with LSTM reaching the highest recorded test accuracy (95.32%). The evidence does not support the README's 98%+ accuracy or Bi-LSTM-best claims, nor does it support the deployed Streamlit application as reliable, production-grade spam detection, given the tokenizer inconsistency and the absence of class-wise metrics. The project is best understood as an instructive comparison of three neural architectures on a small, imbalanced SMS dataset with an unresolved deployment inconsistency.

Future Scope

- Save the training-time tokenizer (e.g., with pickle) and reuse it in app.py so inference matches the training vocabulary.
- Deploy the best-performing model (LSTM) and retrain all models through a single reproducible pipeline with fixed seeds and a notebook kept in sync with the saved model files.
- Compute precision, recall, F1-score and a confusion matrix for every model, and test on genuinely unseen, real-world SMS messages.
- Explore class weighting or oversampling instead of down-sampling, larger and more recent datasets, transformer-based (BERT-family) models, and input validation and error handling in the Streamlit app.

References

[1] T. A. Almeida, J. M. G. Hidalgo, and A. Yamakami, "Contributions to the study of SMS spam filtering: new collection and results," in Proc. 11th ACM Symposium on Document Engineering (DocEng '11), Mountain View, CA, USA, 2011, pp. 259–262.

[2] A. Chandra and S. K. Khatri, "Spam SMS filtering using recurrent neural network and long short term memory," in Proc. 2019 4th Int. Conf. on Information Systems and Computer Networks (ISCON), Mathura, India, 2019, pp. 118–122.

[3] A. L. Rosewelt, N. D. Raju, and S. Ganapathy, "An effective spam message detection model using feature engineering and bi-LSTM," in Proc. 2022 Int. Conf. on Advanced Computing, Communication and Applied Informatics (ACCAI), Jan. 2022, pp. 1–6.

[4] T. Sharma, P. Walia, S. Sharma, and M. Chopra, "NLP-Driven Framework for Spam SMS Detection and Classification," in 2025 3rd World Conference on Communication & Computing (WCONF), Raipur, India, 2025, pp. 1–5.

[5] UCI Machine Learning Repository, "SMS Spam Collection Data Set," Univ. of California, Irvine. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/SMS+Spam+Collection

[6] D. Varma, "SMS-Spam-Detection-using-NLP" (GitHub repository), 2023. [Online]. Available: https://github.com/deepankarvarma/SMS-Spam-Detection-using-NLP — base implementation studied and documented in this report.